

Topic	Questions	Max Marks	Stars
RBMT only	Q2	9	★★
SMT only	Q3	7	★
RBMT + SMT combined	Q1, Q4, Q5	10	★★★★
Cross-lingual translation challenges	Q6	8	—
Natural Language Generation (NLG)	Q7, Q8	9	★★★★★
Question Answering System	Q9, Q10, Q11, Q12	9	★
Sentiment Analysis	Q13	10	★★
Text Entailment	Q15	9	—
Conversational Agents	Q16	8	★★
Text Entailment + Dialog together	Q14	10	—

1. Explain Rule Based Machine Translation and Statistical Machine Translation with suitable diagram and example. [10 Marks] ★★★ Q2. Explain the key principles of rule-based machine translation. How do rule-based techniques differ from statistical approaches? Provide an example of a rule-based translation. [9 Marks] ★★ Q3. Explain Statistical Machine Translation (SMT) with suitable diagrams and example. [7 Marks] ★ Q4. Describe various Machine Translation Approaches. [10 Marks] Q5. Compare the architectures of Rule-Based Machine Translation (RBMT) and Statistical Machine Translation (SMT). How does each approach handle syntactic and semantic ambiguities? [9 Marks]

Statistical Machine Translation (SMT) is a machine translation approach where the system learns to translate by studying millions of example translations — rather than following hand-written grammar rules. It uses probability and statistics to find the most likely translation of a given sentence.

Simple analogy: Imagine you want to learn French by reading 1 million French-English book pairs. You never study grammar rules — you just notice patterns. "Bonjour" always appears where "Hello" appears. "Je t'aime" always maps to "I love you". Over time you learn which French words map to which English words — purely from examples. SMT works exactly this way.

Key idea: Given a sentence in the source language, find the target language sentence that is most **probable** given the source.

Core Idea — Probability Framework

SMT uses **Bayes theorem** to frame translation as a probability problem:

Best Translation = $\text{argmax } P(T | S)$ Find the target sentence T that maximises probability given source sentence S

Using Bayes theorem, this becomes:

$P(T | S) = P(S | T) \times P(T) / P(S)$ Since P(S) is constant for all T:

Best T = $\text{argmax } [P(S | T) \times P(T)]$

This gives us two separate models:

Model	Formula	What it learns
Translation Model	$P(S T)$	Which source words map to which target words
Language Model	$P(T)$	What fluent natural target language looks like

Three Components of SMT

Component 1 — Translation Model $P(S | T)$

Learns which words and phrases in the source language correspond to which words in the target language — from a **parallel corpus** (millions of sentence pairs).

Parallel corpus example: French: "Je suis étudiant" ↔ English: "I am a student" French: "J'aime le cricket" ↔ English: "I love cricket"

From these pairs, the model learns:

- "Je" → "I" with probability 0.9
- "suis" → "am" with probability 0.85
- "étudiant" → "student" with probability 0.92

Component 2 — Language Model $P(T)$

Ensures the translated output is **fluent and natural** in the target language — not just a word-by-word swap.

Trained on a large **monolingual corpus** (only target language text):

"I am a student" → natural English ✓

"Student a am I" → unnatural English ✗

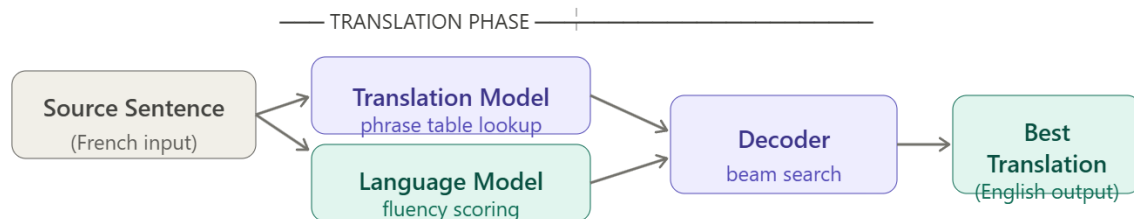
The language model assigns higher probability to natural word orderings.

Component 3 — Decoder

Combines both models to find the **best translation** by searching through all possible target sentences and picking the one with the highest combined probability.

$$\text{Best } T = \text{argmax} [P(S|T) \times P(T)]$$

SMT Full Architecture---



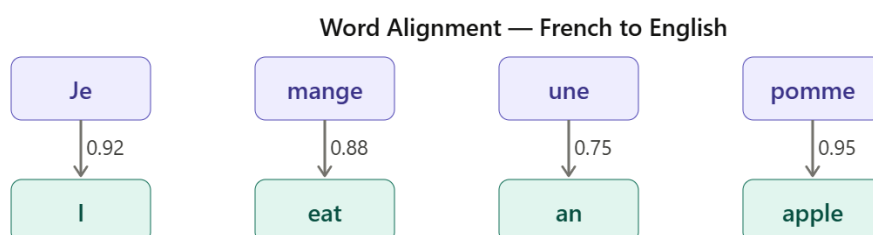
Step-by-Step Example — French to English

Input sentence (French): *"Je mange une pomme"* **Expected output (English):** *"I eat an apple"*

Step 1 — Word Alignment

Learn which French words align to which English words from the parallel corpus:

French	English	Probability
Je	I	0.92
mange	eat	0.88
une	an / a	0.75
pomme	apple	0.95



Step 2 — Phrase Table (Translation Model)

After alignment, the system builds a **phrase table** — mapping phrases (not just individual words) with probabilities:

French phrase English translation P(English | French)

Je	I	0.92
mange	eat	0.88
une pomme	an apple	0.90
Je mange	I eat	0.85
une	a / an	0.75

Note: Phrase-based SMT (PBSMT) works on phrases — more accurate than word-by-word translation.

Step 3 — Language Model Scoring

Candidate translations generated:

Candidate Translation Language Model Score P(T)

"I eat an apple"	0.85 — natural English
"I apple eat an"	0.01 — unnatural
"Eat I apple an"	0.02 — unnatural

Language model picks "I eat an apple" as most fluent.

Step 4 — Decoder Combines Both

Final score = Translation Model $\times$ Language Model:

"I eat an apple": $= P(S|T) \times P(T) = (0.92 \times 0.88 \times 0.90) \times 0.85 = 0.729 \times 0.85 = \mathbf{0.620} \leftarrow$
highest score

Final output: "I eat an apple" ✓

Types of SMT

Type	Description
Word-based SMT	Translates one word at a time — simple but poor quality

Type	Description
Phrase-based SMT (PBSMT)	Translates phrases — much better, most widely used
Syntax-based SMT	Uses parse trees — handles word order changes better
Hierarchical SMT	Combines phrase-based with syntax rules

Advantages and Limitations of SMT

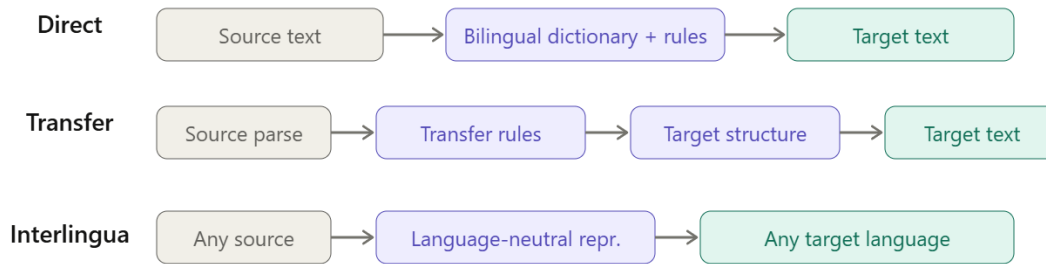
Advantages	Limitations
No hand-written grammar rules needed	Needs millions of parallel sentence pairs
Learns from real translation examples	Poor for low-resource languages
Better than RBMT for complex sentences	Phrase table can become very large
Handles multiple translation options	Cannot handle long-range dependencies well
Improves automatically with more data	Replaced by Neural MT (NMT) in modern systems

Rule-Based Machine Translation (RBMT) is the oldest approach to machine translation where the system translates text by applying **hand-written linguistic rules** — grammar rules, dictionary lookups, morphological rules — created by language experts.

Simple analogy: Imagine learning French by studying a grammar book. The book tells you — "subject comes before verb", "adjectives agree with nouns in gender", "use 'le' for masculine nouns". You follow these rules step by step to construct a French sentence from English. RBMT works exactly this way — the system follows a rulebook written by linguists to convert one language to another.

Key idea: Translation is done by understanding the **structure and grammar** of the source language and applying rules to generate grammatically correct target language output.

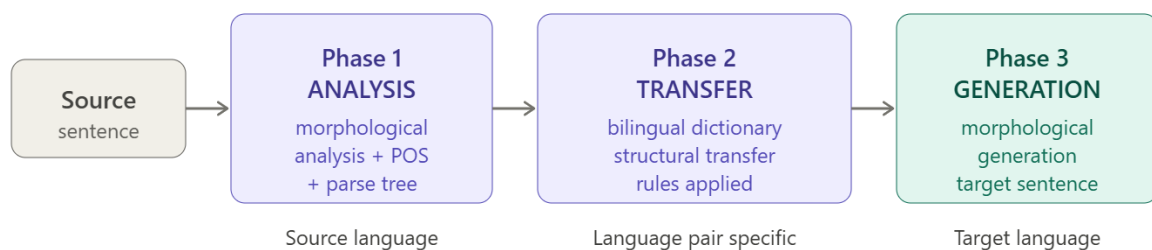
Three Types of RBMT



Type	Approach	Best for
Direct	Word-for-word substitution using bilingual dictionary	Simple sentences, closely related languages
Transfer	Parse source → apply transfer rules → generate target	Most common RBMT approach
Interlingua	Convert to universal meaning representation → generate any language	Multi-language translation systems

Transfer-Based RBMT — Main Approach

The **Transfer-Based** approach is the most widely used RBMT method. It works in three clear phases:---



Step-by-Step Example — English to Hindi

Input sentence (English): "The boy eats a mango." **Expected output (Hindi):** "लड़का आम खाता है।"

Phase 1 — Analysis (Understanding the Source)

The system analyses the English sentence:

Step 1 — Morphological Analysis

Identify the base form and properties of each word:

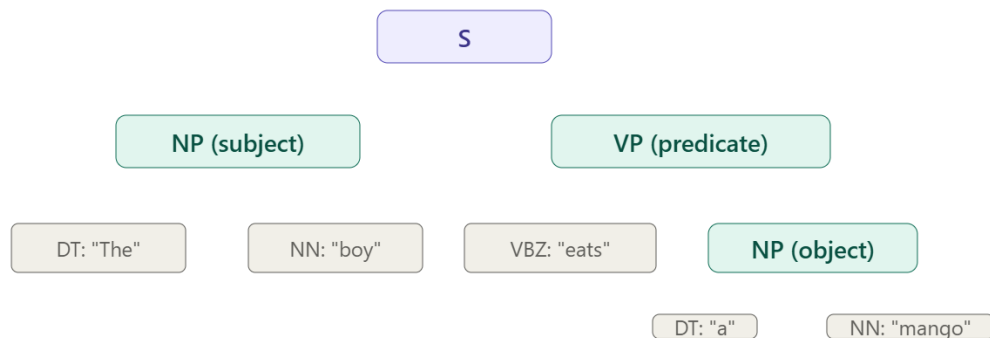
Word	Base form	Type	Properties
------	-----------	------	------------

The	the	Article	Definite
boy	boy	Noun	Singular, Masculine
eats	eat	Verb	3rd person singular, Present
a	a	Article	Indefinite
mango	mango	Noun	Singular

Step 2 — POS Tagging

The(DT) boy(NN) eats(VBZ) a(DT) mango(NN)

Step 3 — Parse Tree---



Phase 2 — Transfer (Applying Rules)

Step 4 — Bilingual Dictionary Lookup

English word Hindi equivalent

boy	लड़का (ladka)
eats	खाता है (khata hai)
mango	आम (aam)

English word Hindi equivalent

The — (no direct equivalent in Hindi)

a — (no direct equivalent in Hindi)

Step 5 — Structural Transfer Rules Applied

Hindi has a **different word order** than English:

English: Subject → Verb → Object (SVO) "*The boy eats a mango*" Subject=boy, Verb=eats, Object=mango

Hindi: Subject → Object → Verb (SOV) "*लड़का आम खाता है*" Subject=लड़का, Object=आम, Verb=खाता है

Transfer rules applied:

Rule	Description
Rule 1 — Word order	Rearrange SVO → SOV for Hindi
Rule 2 — Article drop	Drop English articles "the", "a" (Hindi does not use them)
Rule 3 — Verb agreement	Verb agrees with subject gender — "boy" is masculine → "खाता है"
Rule 4 — Case marking	Add appropriate postposition if needed

Phase 3 — Generation (Producing Target Sentence)

Step 6 — Morphological Generation

Apply target language morphological rules to form the final sentence:

- "boy" → masculine singular → verb form = "खाता है" (correct)
- No articles needed in Hindi
- Add sentence-ending punctuation "।"

Final output: "*लड़का आम खाता है।*" ✓

Advantages and Limitations of RBMT

Advantages

Transparent — rules are readable and explainable

Consistent — same input always gives same output

Works without training data

Good for well-defined language pairs

High precision for simple sentences

Handles grammatical gender and agreement well

Limitations

Creating rules is very expensive and time-consuming

Cannot handle all exceptions and irregular patterns

Does not improve automatically with more data

Poor performance for complex or ambiguous sentences

Requires expert linguists for each language pair

Rules do not scale to handle every real-world sentence

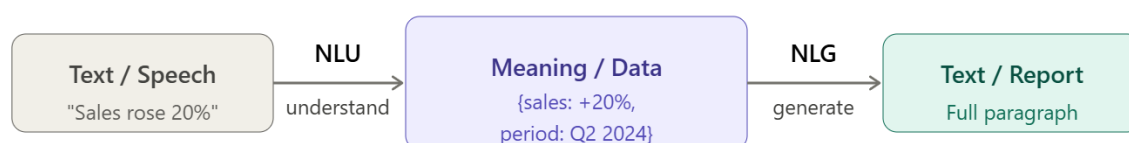
2. Explain Natural Language Generation with reference architecture. [7 Marks] ★★★★★
OR Define natural language generation and its role in NLP. How does NLG differ from text-to-speech synthesis, and what are the applications of NLG in data reporting and storytelling? [9 Marks]

Natural Language Generation (NLG) is the process of automatically producing meaningful, grammatically correct, human-readable text from **structured data, knowledge bases, or internal representations** using NLP techniques.

Simple analogy: Imagine a cricket scorecard with numbers — 248 runs, 6 wickets, 45 overs, Virat Kohli 89 runs. A sports journalist reads these numbers and writes: *"In a thrilling match, India posted 248 runs for 6 wickets in 45 overs, with Virat Kohli anchoring the innings with a solid 89-run knock."* NLG does exactly this — takes raw structured data and writes fluent human-like text automatically, without a journalist.

NLG answers: *"Given this data or knowledge, what should I say and how should I say it?"*

NLG vs NLU — Two Sides of NLP



	NLU (Natural Language Understanding)	NLG (Natural Language Generation)
Direction	Text → Meaning	Meaning/Data → Text
Task	Understand what text means	Write text from data
Input	Human language	Structured data, knowledge
Output	Structured representation	Human-readable text
Example	Parse a customer query	Write a product description

NLG vs Text-to-Speech (TTS) — Key Difference

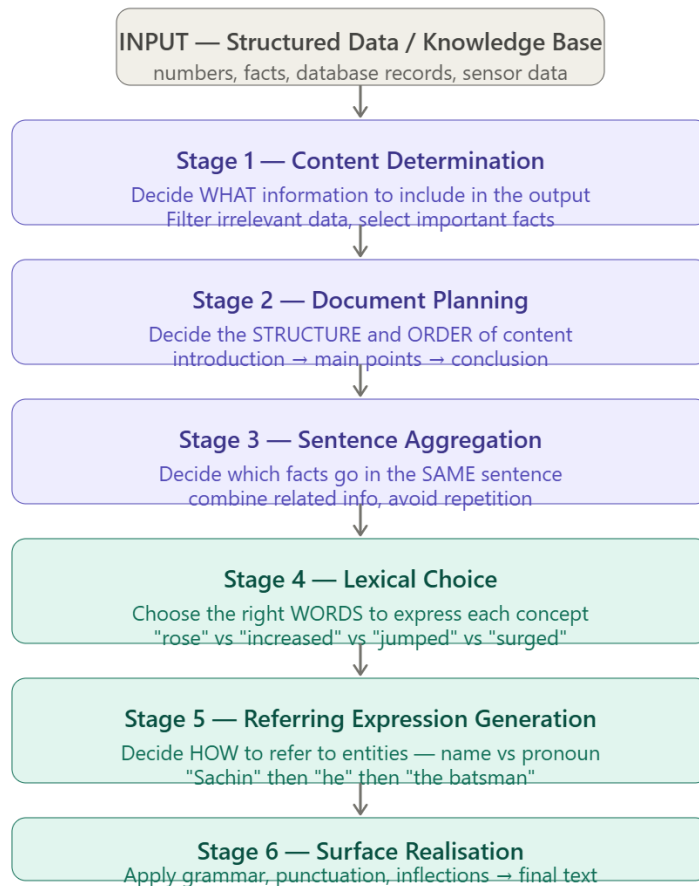
This is a commonly asked comparison. They are completely different processes.

Feature	NLG	Text-to-Speech (TTS)
What it does	Creates NEW text from data	Converts EXISTING text to audio
Input	Structured data, numbers, facts	Already written text
Output	Written text — words, sentences, paragraphs	Audio — spoken voice
Intelligence needed	High — decides WHAT to say and HOW	Lower — only converts text to sound
Example	Generates a weather report paragraph from temperature data	Reads the generated weather report aloud
Works together?	Yes — NLG creates text, TTS speaks it	—

Simple analogy: NLG is like a **scriptwriter** — creates the dialogue from ideas. TTS is like an **actor** — reads the script aloud. The scriptwriter is far more creative; the actor just converts written words to spoken sound.

NLG Reference Architecture — Six Stages

The most widely accepted NLG architecture was proposed by **Reiter and Dale (2000)**. It has **six pipeline stages** — each stage takes the output of the previous one and processes it further.---



Each Stage Explained with Example

We will trace one example through all six stages:

Input data:

```
{
  team: "India",
  score: 248,
  wickets: 6,
  overs: 45,
  top_scorer: "Kohli",
  runs: 89,
  result: "won",
  margin: "34 runs"
}
```

Stage 1 — Content Determination

Decide what information is important enough to include.

Selected facts: team name, score, wickets, top scorer, result, margin Dropped: over-by-over breakdown, fielding stats (not important for summary)

Stage 2 — Document Planning

Decide the order and structure of the output.

Plan:

1. Opening — match result
2. Batting performance — score and top scorer
3. Closing — winning margin

Stage 3 — Sentence Aggregation

Group related facts into sentences — avoid separate sentences for every fact.

Combine: {score=248, wickets=6, overs=45} → one sentence about batting
Combine: {top_scorer=Kohli, runs=89} → one sentence about performance

Stage 4 — Lexical Choice

Choose the most appropriate words for each concept.

"won" → "triumphed", "clinched victory"
"248 runs" → "posted 248", "put up 248"
"scored 89" → "anchored innings with 89", "contributed a vital 89"

Stage 5 — Referring Expression Generation

Decide how to refer to entities across sentences.

First mention: "India", "Virat Kohli"
Later: "the team", "he", "the star batsman"
Avoids repeating "India" and "Virat Kohli" every sentence

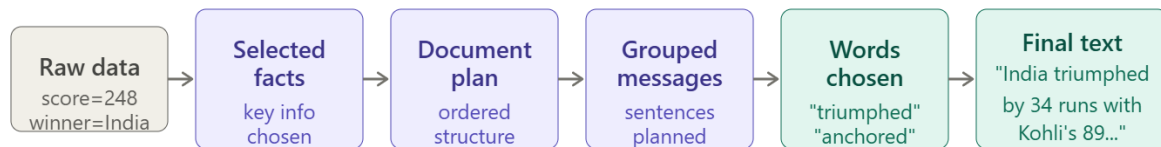
Stage 6 — Surface Realisation

Apply grammar rules, tense agreement, punctuation, and produce final text.

Final generated output:

"India triumphed in a closely fought contest, clinching victory by 34 runs. The team posted a competitive 248 for 6 in their 45 overs, with Virat Kohli anchoring the innings brilliantly with a composed knock of 89 runs."

NLG Architecture — Diagram with Data Flow---



Each stage transforms the representation — from raw numbers to fluent human text

Applications of NLG

Application 1 — Automated Data Reporting (Most Important)

NLG converts raw data tables and statistics into full written reports automatically.

Example: Automated Insights' **Wordsmith** platform reads financial data from Excel and generates: *"Revenue for Q2 2024 grew 18% year-on-year to Rs 2,450 crore, driven by strong performance in the digital segment which recorded a 34% surge."*

This used to take a financial analyst 3 hours — NLG does it in 3 seconds for thousands of companies simultaneously.

Application 2 — Sports Commentary and Storytelling

Example: The Associated Press uses NLG to automatically generate match reports for thousands of cricket, football, and baseball matches — generating articles that are indistinguishable from human-written sports journalism.

"India posted 248/6 in their 45 overs, with Virat Kohli's composed 89 setting the platform before late flourishes from Hardik Pandya and Ravindra Jadeja guided India to a defendable total."

Stage	Question answered	Output
Content Determination	What to say?	Selected relevant facts

Stage	Question answered	Output
Document Planning	In what order?	Structured content plan
Sentence Aggregation	How to group?	Sentence-level groupings
Lexical Choice	Which words?	Words and phrases selected
Referring Expression	How to reference entities?	Pronouns and descriptions chosen
Surface Realisation	Final grammar?	Grammatically correct text

3. Explain Question Answering system with neat diagram. [7 Marks] ★ Q10. Explain three stages of the Question Answering system with neat diagram. [7 Marks] Q11. Explain the architecture and challenges of building a Question Answering (QA) system. [9 Marks] Q12. Explain the fundamental components of a question-answering system. How do QA systems utilize machine learning and NLP techniques to find accurate answers? [8 Marks]

A **Question Answering (QA) system** is an NLP system that automatically finds and returns a **precise, direct answer** to a question asked in natural language — rather than returning a list of documents like a search engine.

Simple analogy: When you type "What is the capital of France?" into Google, it shows you ten links. A QA system directly answers "**Paris**" — no links, no reading required. It is like having a knowledgeable assistant who reads everything for you and gives you the exact answer.

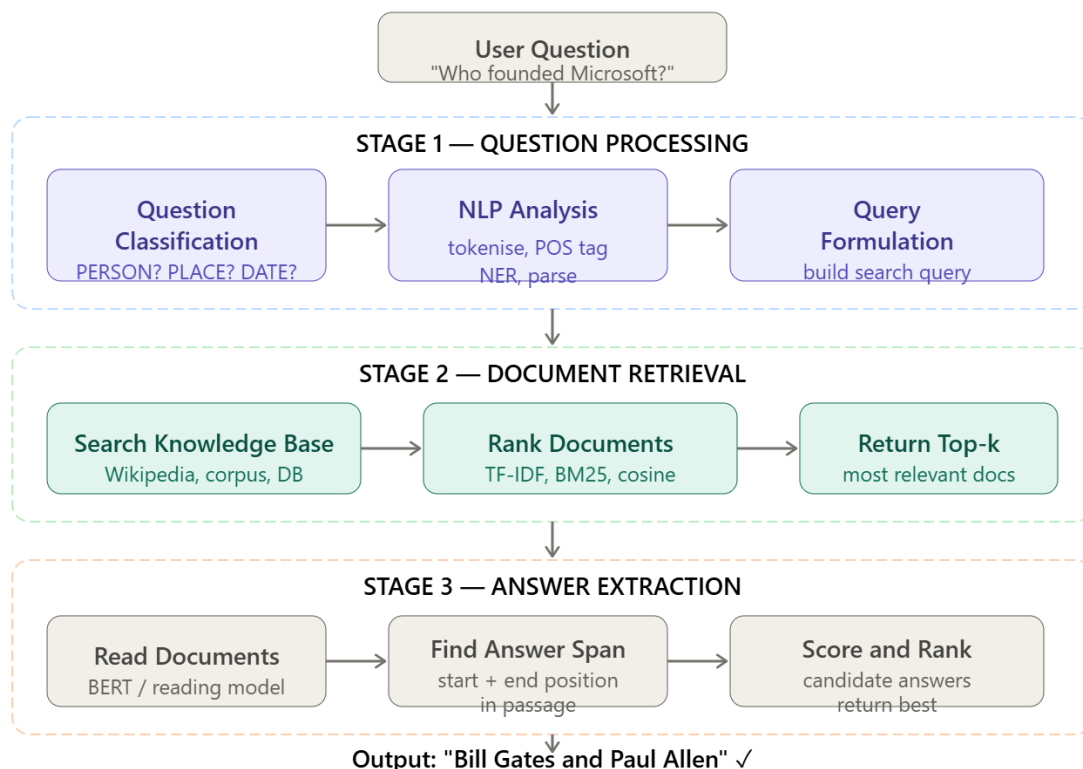
Search engine: Returns documents that may contain the answer **QA system:** Returns the exact answer itself

Types of QA Systems

Type	Description	Example
Factoid QA	Answers simple fact-based questions	"Who founded Apple?" → Steve Jobs
List QA	Returns a list of answers	"Name 5 planets"
Definition QA	Explains what something is	"What is NLP?"
Conversational QA	Multi-turn dialogue	Chatbots, Siri

Type	Description	Example
Open-domain QA	Searches any topic	IBM Watson, ChatGPT
Closed-domain QA	Specific domain only	Medical QA, Legal QA

QA System Architecture — Full Diagram---



Three Stages of QA System:

Stage 1 — Question Processing

This stage analyses and understands the user's question before searching for an answer.

Step 1 — Question Classification

Classify the question by the **type of answer expected**:

Question type Expected answer type Example

WHO	PERSON	"Who founded Google?"
WHEN	DATE/TIME	"When was India founded?"

Question type Expected answer type Example

WHERE	LOCATION	"Where is the Taj Mahal?"
WHAT	DEFINITION/THING	"What is photosynthesis?"
HOW MANY	NUMBER	"How many planets are there?"
WHY	REASON	"Why is the sky blue?"

Step 2 — NLP Analysis

Apply standard NLP processing to the question:

Question: "Who founded Microsoft in 1975?"

Tokenise → ["Who", "founded", "Microsoft", "in", "1975"]

POS tag → [WP, VBD, NNP, IN, CD]

NER → Microsoft=ORG, 1975=DATE

Parse → subject=Who, verb=founded, object=Microsoft

Key terms → ["founded", "Microsoft", "1975"]

Step 3 — Query Formulation

Build a search query from the extracted key terms:

Query: "Microsoft founded 1975 founder"

Stage 2 — Document Retrieval

This stage searches for documents that are likely to contain the answer.

Step 1 — Search the Knowledge Source

The knowledge source can be:

- **Wikipedia** — general open-domain QA
- **Domain corpus** — medical, legal, scientific documents
- **Database** — structured data like company records
- **Web** — real-time web search

Step 2 — Rank Documents by Relevance

Use IR techniques to rank all matching documents:

Method	How it works
TF-IDF	Score documents by term frequency $\times$ inverse document frequency
BM25	Advanced version of TF-IDF — better ranking
Cosine similarity	Measure angle between query and document vectors
Dense retrieval	Use BERT embeddings — semantic matching

Step 3 — Return Top-k Documents

Return the top 5-10 most relevant documents (passages) for the answer extraction stage.

Retrieved passage: *"Microsoft Corporation was founded by Bill Gates and Paul Allen on April 4, 1975, to develop and sell BASIC interpreters for the Altair 8800."*

Stage 3 — Answer Extraction

This stage reads the retrieved documents and extracts the precise answer span.

Step 1 — Read and Understand Passages

A reading comprehension model (typically BERT-based) reads each retrieved passage alongside the original question.

Step 2 — Find Answer Span

The model identifies the **start and end positions** of the answer within the passage:**Step 3 — Score and Return Best Answer**

Candidate 1: "Bill Gates and Paul Allen" — score 0.93 ← selected Candidate 2: "April 4, 1975" — score 0.41 (date, not person) Candidate 3: "Microsoft Corporation" — score 0.22

Final answer returned: *"Bill Gates and Paul Allen"*

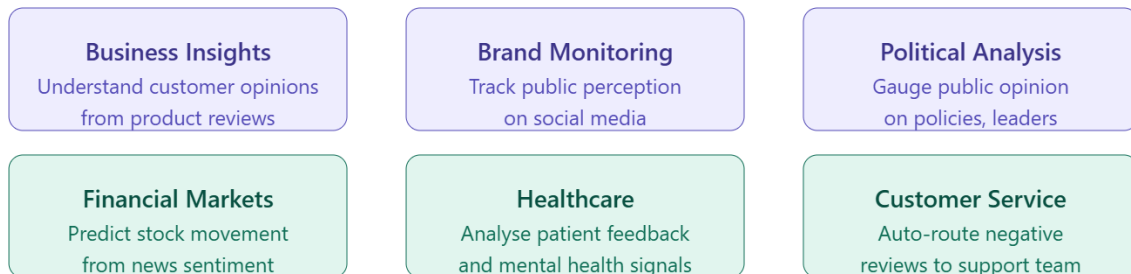
4. Write a note on Sentiment Analysis. [10 Marks] ★★

Sentiment Analysis (also called **Opinion Mining**) is an NLP task that automatically identifies and classifies the **emotional tone, opinion, or attitude** expressed in a piece of text — whether it is positive, negative, or neutral.

Simple analogy: Imagine a restaurant owner who receives 10,000 customer reviews every month. Reading each one manually is impossible. Sentiment Analysis acts like an intelligent assistant that reads all 10,000 reviews in seconds and reports — *"6,500 are positive, 2,800 are negative, 700 are neutral. Most complaints are about slow service and most praise is about food quality."* This gives the owner actionable insights instantly.

Input: *"The food was absolutely amazing but the service was terribly slow."* **Output:** Mixed sentiment — Positive about food, Negative about service

Why is Sentiment Analysis Important?---



Levels of Sentiment Analysis

Sentiment analysis can be performed at different levels of granularity:

Level	What it analyses	Example
Document level	Overall sentiment of entire document	Full review = Positive
Sentence level	Sentiment of each sentence	Sentence 1 = Positive, Sentence 2 = Negative
Aspect level	Sentiment towards specific aspects	Food = Positive, Service = Negative
Word level	Polarity of individual words	"amazing" = +1, "terrible" = -1

Aspect-level example:

"The battery life is excellent but the camera is disappointing." Battery → Positive | Camera → Negative

Types of Sentiment Classification

Type 1 — Polarity Classification

Most common — classifies sentiment into categories:

Class	Meaning	Example
Positive	Favourable opinion	"I love this product!"
Negative	Unfavourable opinion	"This is terrible quality."

Class	Meaning	Example
Neutral	No strong opinion	"The product arrived on time."

Type 2 — Fine-grained Sentiment

More detailed scale — 5 classes like star ratings:

Very Negative → Negative → Neutral → Positive → Very Positive (1 star) (2 stars) (3 stars) (4 stars) (5 stars)

Type 3 — Emotion Detection

Classifies specific emotions rather than just polarity:

Joy | Sadness | Anger | Fear | Surprise | Disgust

"I can't believe they cancelled the event!" → Anger + Surprise

Evaluation Metrics

Metric	Formula	Meaning
Accuracy	Correct / Total	Overall correctness
Precision	$TP / (TP + FP)$	When it says positive, how often correct
Recall	$TP / (TP + FN)$	How many positives were caught
F1 Score	$2 \times P \times R / (P + R)$	Balance of precision and recall

Real-World Applications

Application	How Sentiment Analysis is Used
Product reviews (Amazon, Flipkart)	Automatically summarise customer opinions for sellers
Social media monitoring (Twitter)	Track brand reputation in real time
Financial trading	Analyse news sentiment to predict stock movements
Political campaigns	Gauge public reaction to speeches and policies

Application	How Sentiment Analysis is Used
Movie reviews	Aggregate critic and audience sentiment
Customer support	Auto-escalate negative feedback to human agents
Healthcare	Detect depression or mental health issues from text

5. Discuss the key components of a conversational agent (chatbots or virtual assistants). How do NLG and NLU play a role in creating effective conversational agents? [8 Marks] ★★

A **Conversational Agent** is a software system that can **understand human language and respond intelligently** in a natural, human-like way — through text or speech. Chatbots and virtual assistants are the most common examples.

Simple analogy: A conversational agent is like a **very patient, always-available customer service representative** who never gets tired, never gets angry, can handle thousands of conversations simultaneously, and always gives consistent responses. The difference is that this representative is entirely made of software — it reads your message, understands what you want, thinks of the best response, and replies — all within milliseconds.

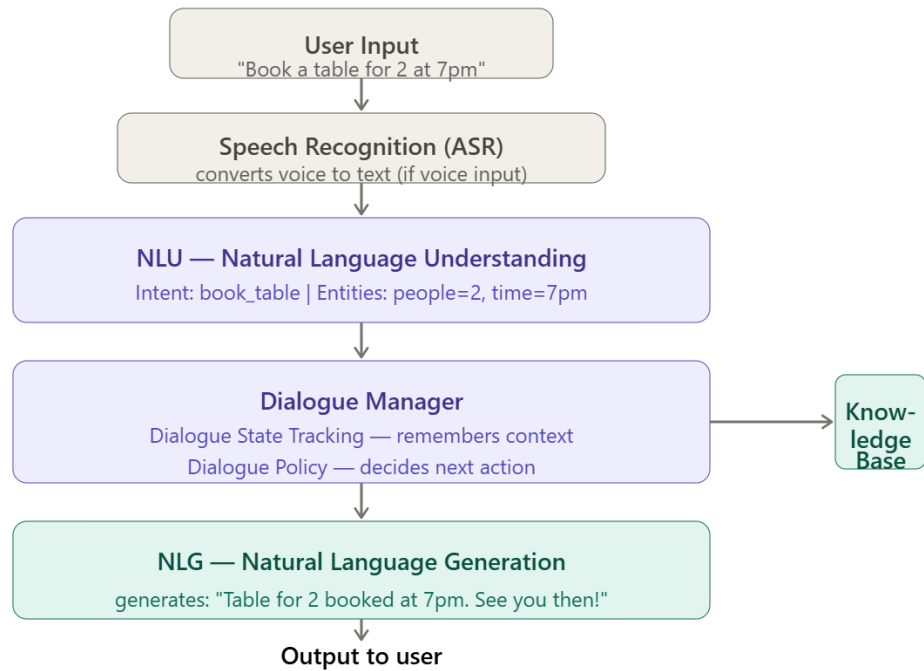
Examples:

- **Chatbots** — customer support bots on websites, Telegram bots
- **Virtual Assistants** — Siri (Apple), Alexa (Amazon), Google Assistant, Cortana

Types of Conversational Agents

Type	How it works	Example
Rule-based	Follows if-then rules and decision trees	Simple FAQ chatbots
Retrieval-based	Picks best response from pre-written answers	Customer support bots
Generative	Generates new responses using deep learning	ChatGPT, Gemini
Task-oriented	Focused on completing specific tasks	Booking, ordering, banking
Open-domain	Handles any topic in general conversation	GPT-4, Claude

Key Components of a Conversational Agent---



Each Component — Detailed Explanation

Component 1 — Automatic Speech Recognition (ASR)

Converts spoken voice input into text. This is the entry point for voice-based assistants like Siri and Alexa.

User says: *"Hey Alexa, play some jazz music"* ASR converts to text: *"Hey Alexa play some jazz music"*

Used in: Alexa, Google Assistant, Siri Not needed for: text-based chatbots

Component 2 — NLU (Natural Language Understanding)

The most critical input-side component. NLU reads the text and extracts **structured meaning** from it by performing two key tasks:

Task A — Intent Detection: Identifies WHAT the user wants to do.

User message	Detected intent
"Book a flight to Mumbai"	book_flight
"What is the weather today?"	get_weather
"Cancel my order"	cancel_order
"Tell me a joke"	get_joke

Task B — Entity Extraction (Slot Filling): Identifies the SPECIFIC DETAILS of the request.

"Book a flight to Mumbai on Monday for 2 people" Intent: book_flight Entities: destination=Mumbai, date=Monday, passengers=2

NLU uses NER, POS tagging, and dependency parsing to extract these entities.

Component 3 — Dialogue Manager

The brain of the conversational agent. It has two sub-components:

3A — Dialogue State Tracker (DST)

Maintains the **context of the entire conversation** — remembers what was said earlier.

Turn 1: User: "Book a table for 2"

State: {intent: book_table, people: 2, time: None}

Turn 2: User: "At 7pm"

State: {intent: book_table, people: 2, time: 7pm}

← DST remembered "people: 2" from Turn 1

Without DST, the bot would forget what was discussed in previous turns.

3B — Dialogue Policy

Decides WHAT ACTION to take next based on the current state:

State	Policy decision
Missing information	Ask a clarifying question
All info collected	Execute the task
User is confused	Offer help options
Task completed	Confirm and close

Component 4 — Knowledge Base / Backend

Stores the actual information and data the agent needs to answer questions or complete tasks.

Type	Example	Used for
Database	Restaurant availability, flight schedules	Task completion
FAQ database	Pre-written answers	FAQ bots
Knowledge graph	Entity relationships	General QA
External API	Weather, maps, stock prices	Real-time data

Component 5 — NLG (Natural Language Generation)

Takes the structured response from the dialogue manager and generates a **natural, fluent human-readable reply**.

Dialogue manager output: {status: booked, people: 2, time: 7pm, table: 4} NLG generates:
"Great news! I've booked table 4 for 2 people at 7:00 PM. We look forward to seeing you!"

NLG ensures responses are:

- Grammatically correct
- Natural and conversational in tone
- Personalised to the context
- Appropriately polite or friendly

Component 6 — Text-to-Speech (TTS)

Converts the text response back to spoken audio for voice-based assistants.

NLG output: "Table booked for 7pm" TTS output: Natural spoken voice saying the sentence

Used in: Alexa, Siri, Google Assistant Not needed for: text chatbots

Role of NLU in Conversational Agents

NLU is responsible for:

- **Intent detection** — classifying what the user wants
- **Entity extraction** — finding specific details (dates, names, locations)
- **Coreference resolution** — understanding "it", "there", "him" in context
- **Sentiment detection** — detecting if user is frustrated or satisfied
- **Handling paraphrases** — "book a table" = "reserve a spot" = "make a reservation"

Without accurate NLU, the bot cannot understand the user — it would fail at the very first step.

Role of NLG in Conversational Agents

NLG is responsible for:

- **Generating natural responses** — not robotic or template-sounding text
- **Personalisation** — adapting tone based on user (formal for business, friendly for casual)
- **Handling context** — using pronouns, avoiding repetition across turns
- **Multi-turn coherence** — responses make sense in the flow of conversation
- **Appropriate length** — brief for simple queries, detailed for complex ones

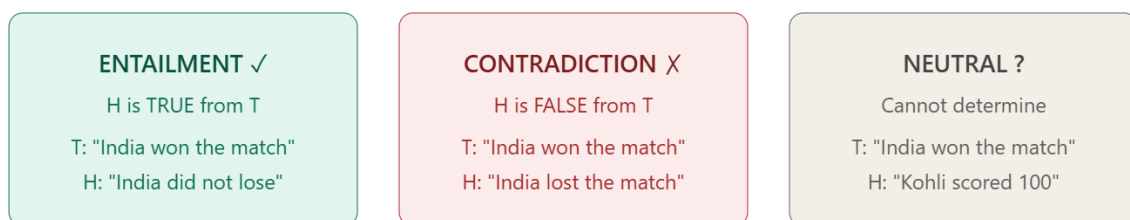
Without NLG, responses would be rigid templates — *"BOOKING CONFIRMED. TIME: 7PM. TABLE: 4"* — which feels robotic and unfriendly.

6. Describe following NLP applications: (i) Text Entailment (ii) Dialog and Conversational Agents. [10 Marks]

Text Entailment is an NLP task that determines whether a **Hypothesis (H)** can be logically inferred from a **Premise (T)**.

Simple analogy: You read *"It is raining heavily outside."* Your friend says *"You will need an umbrella."* — Does the first statement logically lead to the second? Yes. That logical checking is text entailment.

Three Possible Outcomes---



Text Entailment vs Text Similarity vs Paraphrasing

Feature	Text Entailment	Text Similarity	Paraphrasing
Direction	One-way ($T \rightarrow H$)	Symmetric	Symmetric ($A \leftrightarrow B$)

Feature	Text Entailment	Text Similarity	Paraphrasing
What it checks	Logical inference	Degree of closeness	Same meaning, different words
Output	Entail / Contradict / Neutral	Score 0 to 1	Yes / No
Example	"Dog bit man" → "Man was bitten"	"car" ≈ "automobile"	"She bought a dress" ↔ "A dress was bought by her"

Key point: Entailment is **asymmetric** — T entails H does NOT mean H entails T. "The dog bit the man" entails "A man was bitten" — but not vice versa (could be any animal).

Importance of Text Entailment

- **Question Answering** — verify if retrieved passage entails the correct answer
- **Fake News Detection** — check if article body entails the headline claim
- **Text Summarisation** — ensure summary does not contradict original
- **Information Extraction** — verify extracted facts are supported by source text
- **Machine Translation** — check if translation preserves original meaning

(ii) Dialog and Conversational Agents

A **Conversational Agent** (Dialog System) is an NLP application that can **understand human language and respond intelligently** in natural, multi-turn conversation — through text or speech.

Simple analogy: A conversational agent is like a **patient, always-available assistant** who reads your message, understands exactly what you need, fetches the right information, and responds naturally — handling thousands of such conversations simultaneously.

Key Components



Component	Role
ASR	Converts voice to text (for voice assistants)
NLU	Detects intent + extracts entities from user message
Dialogue Manager	Tracks conversation state, decides next action
Knowledge Base	Stores facts, product data, answers
NLG	Generates natural, fluent response
TTS	Converts text back to speech (optional)

Multi-Turn Example

Turn	User	Bot
1	"Book a hotel in Goa"	"What are your check-in and check-out dates?"
2	"15th to 20th December"	"How many guests?"
3	"2 adults"	"Sea View Resort found for Rs 4,500/night. Shall I confirm?"
4	"Yes please"	"Booked! Confirmation sent to your email."

The bot remembers all details across turns — this is **dialogue state tracking**.

7. Define text entailment and describe its importance in natural language understanding. How does text entailment differ from text similarity or paraphrasing? Provide an example. [9 Marks]

Text Entailment (also called **Natural Language Inference** — NLI or **Recognising Textual Entailment** — RTE) is an NLP task that determines whether a given statement called the **Hypothesis (H)** can be logically concluded or **inferred** from another statement called the **Premise (T)**.

Simple analogy: You read a news article — *"The government announced free education for all children below 14 years."* Your friend says — *"Children will not have to pay school fees."* Now you need to decide — does the article support your friend's claim? Yes, it logically follows. That process of checking whether one statement can be derived from another is exactly what Text Entailment does — automatically.

Premise (T): What we already know — the source text **Hypothesis (H):** The statement we want to verify **Question asked:** Does T logically support H?

Three Possible Outcomes---

ENTAILMENT H is TRUE given T T logically supports H T: "The school won the championship" H: "The school competed in a tournament" Result: YES ✓	CONTRADICTION H is FALSE given T H directly opposes T T: "The school won the championship" H: "The school lost the championship" Result: NO ✗	NEUTRAL Cannot determine T does not confirm H T: "The school won the championship" H: "The coach was very happy" Result: UNKNOWN ?
--	--	---

Worked Example — Step by Step

Input pair:

Premise: *"The doctor gave the patient a prescription for antibiotics after diagnosing bacterial infection."* **Hypothesis:** *"The patient was diagnosed with an illness."*

Step 1 — Understand the premise: Doctor gave prescription → patient diagnosed → bacterial infection

Step 2 — Check if hypothesis follows: "Bacterial infection" is an illness → "patient was diagnosed with an illness" → directly supported

Step 3 — Determine outcome:

Result: ENTAILMENT ✓ The premise explicitly states diagnosis of bacterial infection which is an illness.

Now try a different hypothesis:

Hypothesis: *"The patient recovered completely after taking medicine."* **Result: NEUTRAL**
Premise only says prescription was given — not whether patient recovered.

Importance of Text Entailment in NLP

Text entailment is considered one of the most important benchmarks for measuring **true language understanding**. A system that can correctly determine entailment has demonstrated it can reason about meaning — not just match words.



Why it matters for language understanding:

- **Goes beyond keyword matching** — a system must understand the actual meaning to decide entailment
- **Tests logical reasoning** — requires understanding negation, quantifiers, implication
- **Universal benchmark** — models that perform well on NLI (Stanford SNLI dataset) demonstrate genuine language comprehension
- **Foundation for higher tasks** — QA, summarisation, and fact checking all depend on entailment reasoning

How Text Entailment Differs from Text Similarity and Paraphrasing

This is the most important part of the question — understanding these three distinct concepts.

Concept 1 — Text Entailment

Checks: *"Can I logically conclude H from T?"* Direction: **One-way** — $T \rightarrow H$ (asymmetric)

Output: Entailment / Contradiction / Neutral

Concept 2 — Text Similarity

Checks: *"How semantically close are these two texts?"* Direction: **Both ways** (symmetric — if A is similar to B, B is similar to A) Output: A similarity score between 0 and 1

Concept 3 — Paraphrasing

Checks: *"Do these two texts express exactly the same meaning?"* Direction: **Both ways** (symmetric — if A paraphrases B, B paraphrases A) Output: Yes / No — same meaning in different words

More Examples Clarifying All Three

Example set — Same sentences, different tasks:

A: "All students passed the final examination." B: "Some students passed the final examination."

- **Entailment (A→B):** YES — if ALL passed, then SOME passed ✓
- **Entailment (B→A):** NO — if SOME passed, does not mean ALL passed ✗
- **Paraphrase:** NO — they mean different things
- **Similarity score:** Moderate — same topic, different quantifier

This perfectly shows why entailment is **not the same as similarity or paraphrasing**.

Challenges in Text Entailment

Challenge	Why it is hard	Example
Negation	"He did not fail" entails "He passed" — negation changes meaning completely	Hard for simple models
Quantifiers	"Some" vs "All" vs "None" — subtle but critical difference	"Some passed" ≠ "All passed"
World knowledge	Some entailments need real-world facts not in text	"Water boils at 100°C" → "It was very hot"
Coreference	"John told Peter he was late. He was angry." — who is angry?	Ambiguous pronoun
Long sentences	Complex nested clauses make inference harder	Multi-clause legal texts